Sankhya — the complete account

From the mathematics up: what is being built, how it works, what was rejected and why

Mohit Prajapati

30 August 2026

Contents

Part I — The problem

1. What MRPL actually asked for

Problem statement SIH26119: an **indigenous, GPU-accelerated optimization solver** for refinery planning.

Unpack that into what it means to build.

A refinery buys several crude oils, runs them through units (distillation, reformer, cracker), and blends the intermediate streams into products — petrol, diesel, jet fuel. Every decision is a number: how many barrels of Bombay High to run, how much naphtha goes to the reformer instead of into the gasoline pool, how much of each stream ends up in each product.

Those numbers are not free. Each unit has a capacity. Every stream that enters a unit must leave it (material balance). Every product has specifications it must meet — octane, sulphur, density. And you want the combination that makes the most margin.

Written down, that is exactly a **linear program**, and the refinery industry has known this since the 1950s. What changes is scale and speed: a real planning model has tens of thousands of constraints, and planners want to re-run it many times a day with different crude prices.

"Indigenous" means: do not wrap Gurobi or CPLEX. Build the solver.

"GPU-accelerated" means: it has to actually use the hardware, which turns out to constrain the choice of algorithm heavily — more on that in Part III.

2. Three problem classes, and why all three are needed

Linear program (LP). Everything linear.

$$\min_x \; c^\top x \quad \text{subject to} \quad l \le Ax \le u, \quad lo \le x \le hi$$

x is the vector of decisions, c the cost of each, A the constraint matrix. This is the core planning problem.

Mixed-integer program (MILP). The same, but some variables must be whole numbers.

$$x_j \in \mathbb{Z} \quad \text{for } j \in I$$

Needed the moment a decision is *yes or no* rather than *how much*: run this unit or not, buy this crude cargo or not, use this blending recipe or not. Also for anything piecewise-linear, which is how a nonlinear blending curve gets approximated.

Quadratic program (QP). A squared term in the objective.

$$\min_x \; \tfrac{1}{2} x^\top Q x + c^\top x$$

Needed when you are penalising deviation — "stay close to last month's plan", "minimise the squared error against a target blend". Risk terms are quadratic too.

A planning tool that only does LP is a demo. A refinery needs at least LP and MILP, and QP the moment anything is a least-squares fit.

Part II — The mathematics you need

This part is the whole background. If you know LP duality, skip to Part III.

3. Why a linear program has a corner solution

The feasible region $\{x : l \le Ax \le u,\ lo \le x \le hi\}$ is an intersection of half-spaces — a **polyhedron**. The objective $c^\top x$ is linear, so its contours are parallel hyperplanes.

Push a hyperplane in the direction $-c$ as far as it will go while still touching the polyhedron. The last point of contact is either a **vertex**, or a whole face (if c happens to be parallel to it) — and if it is a face, one of its vertices is also optimal.

Consequence: there is always an optimal solution at a vertex. You never have to search the interior. That single fact is what the simplex method is built on.

4. Duality — the idea the whole solver rests on

This is the most important concept in the document. Everything — how the simplex knows it is finished, how branch-and-bound prunes, what a shadow price is, why presolve can fix a variable — is duality.

4.1 The idea

Take the LP in the form

$$\min_x c^\top x \quad \text{s.t.} \quad Ax \ge b, \quad x \ge 0$$

Suppose I claim the optimum is at least 50. How would I prove it to you without solving the problem?

Take a non-negative combination of the constraints. Pick weights $y \ge 0$ and add up the rows:

$$y^\top Ax \ge y^\top b$$

Now, if it happened that $y^\top A \le c^\top$ componentwise, then for any feasible $x \ge 0$:

$$c^\top x \ge y^\top Ax \ge y^\top b$$

So $y^\top b$ is a **lower bound on the optimum**, and I proved it just by producing y. Naturally I want the best such bound, which is another LP:

$$\max_y \ b^\top y \quad \text{s.t.} \quad A^\top y \le c, \quad y \ge 0$$

That is the **dual**. The original is the **primal**.

4.2 The two theorems

Weak duality. Any feasible y gives a bound: $b^\top y \le c^\top x$ for every feasible pair. This is the derivation above, and it needs no assumptions.

Strong duality. At the optimum the two are *equal*: $b^\top y^\star = c^\top x^\star$. The best bound is exactly the answer.

Strong duality is why a solver can ever say "this is optimal, and here is the proof". It produces both x and y, checks that their objectives match, and the matching is the certificate.

4.3 Reduced costs and complementary slackness

Define the **reduced cost** of variable j:

$$d_j = c_j - (A^\top y)_j$$

Read it as: the true cost of x_j, minus what the constraints will pay you for using it. If $d_j > 0$, increasing x_j makes the objective worse. If $d_j < 0$, increasing it helps.

At an optimum, **complementary slackness** holds:

- if $x_j > lo_j$ strictly, then $d_j \le 0$ is impossible — the variable would want to go up further; so $d_j = 0$ or it sits at a bound
- if $d_j > 0$, then x_j sits at its **lower** bound
- if $d_j < 0$, then x_j sits at its **upper** bound

That is the optimality test, and it is exactly what the solver checks before claiming optimality. Section 14 records what happened the one time it did not.

4.4 What a dual value *means* to a refinery

For a capacity row, y_i is the **shadow price**: how much more margin you would make per extra unit of that capacity.

If the crude distillation unit has $y_i = 525$, then one more barrel per day of CDU capacity is worth Rs 525/day of margin. That is a direct answer to "where should we spend capital", and it comes out of the solve for free.

This is why the solver goes to real trouble to recover dual values through presolve — and says so when the recovery is only approximate. A shadow price you cannot trust is worse than none.

5. Why integrality makes it hard

Add $x_j \in \mathbb{Z}$ and the feasible set stops being a polyhedron. It becomes a lattice of points inside one, and the corner argument of Section 3 collapses — the optimal integer point need not be at a vertex of the relaxation, and usually is not.

There is no known algorithm that solves this in polynomial time in general; MILP is NP-hard. What is done instead:

1. **Relax.** Drop the integrality and solve the LP. Its optimum is a *bound* — for a minimisation, the true integer optimum can only be higher.
2. **Branch.** Pick a variable that came out fractional, say $x_j = 3.4$. Any integer solution has either $x_j \leq 3$ or $x_j \geq 4$. Make two subproblems.
3. **Bound and prune.** If a subproblem's relaxation bound is already worse than the best integer solution found so far, nothing inside it can beat that solution. Discard it without exploring.
4. Repeat.

The whole game is in step 3. A better bound prunes more. That is why cuts, presolve and good branching matter so much — every one of them is a way of making the bound tighter so more of the tree disappears.

And it is why a wrong bound is catastrophic. If a bound is too high by even a rounding error, step 3 discards a subtree that contained the answer, and the solver then proves an optimum that is not one. The `fiber` failure in Section 14 is exactly this.

Part III — The methods, and why these ones

6. Three ways to solve an LP

	how it works	strength	weakness
Simplex	walk vertex to vertex	exact; warm starts	sequential, hard to parallelise
Interior point	cut through the middle	fast on large problems	needs a sparse factorisation each step; no warm start
First-order	matrix-vector products only	massively parallel; GPU	slow to high accuracy

This project implements **simplex** and **first-order**. Section 13.1 gives the full argument for why interior point was left out, from both sides.

7. The simplex method, concretely

Write the LP with slacks so that $Ax = b$, $x \geq 0$, with A having m rows.

Choose m columns to form an invertible **basis** B; the rest are nonbasic and sit at a bound. Then

$$x_B = B^{-1}b$$

and the nonbasic variables are determined by which bound they are at.

One iteration:

1. **Price.** Compute duals $y^\top = c_B^\top B^{-1}$ and reduced costs $d_j = c_j - y^\top A_j$ for nonbasic j. If every d_j has the right sign (Section 4.3), we are optimal — stop.
2. **Choose an entering variable** q with a favourable d_q.
3. **Ratio test.** Increasing x_q changes the basics along $\alpha_q = B^{-1}A_q$. Increase x_q until the first basic variable hits a bound. That variable leaves.
4. **Pivot.** Swap them, update B^{-1}.

Three things dominate the cost, and each has a name in the code:

- **FTRAN** — solving $B\alpha = A_q$
- **BTRAN** — solving $B^\top y = c_B$
- **PRICE** — forming d_j for every nonbasic column

7.1 The ratio test, written out

This is where correctness lives, so it is worth the algebra. Increasing the entering variable by $t \geq 0$ moves the basics:

$$x_B(t) = x_B - t\,\alpha_q, \qquad \alpha_q = B^{-1}A_q$$

Each basic i stays inside $[lo_i, hi_i]$ only while

$$t \le \begin{cases} \dfrac{x_{B_i} - lo_{B_i}}{\alpha_{qi}} & \alpha_{qi} > 0 \quad \text{(falling toward its lower bound)} \\ \dfrac{x_{B_i} - hi_{B_i}}{\alpha_{qi}} & \alpha_{qi} < 0 \quad \text{(rising toward its upper bound)} \end{cases}$$

The smallest such t is the step; the argmin is the leaving variable r. If no α_{qi} has the blocking sign, t is unbounded and so is the LP.

Two practical points that are not decoration:

- **Pivot magnitude matters more than step length.** The chosen α_{qr} becomes a divisor in the basis update; picking a tiny one because it wins the ratio by 10^{-12} poisons the factorisation. The implementation takes the best pivot among candidates within a tolerance of the minimum ratio.
- **Degeneracy.** When x_{B_i} already sits on its bound the ratio is 0, the step is 0, and the objective does not move. Repeated, this cycles. The EXPAND scheme allows a tiny, slowly growing bound relaxation so a strictly positive step is always available.

7.2 Devex, written out

Dantzig picks $\arg\max_j |d_j|$. That is the steepest *slope*, but the true descent per unit distance moved is $d_j / \|\alpha_j\|$, and computing every α_j is the thing we are trying to avoid.

Devex keeps a reference framework and an approximate weight $w_j \approx \|\alpha_j\|^2$, and prices on

$$\text{score}_j = \frac{d_j^2}{w_j}$$

After a pivot on (r, q), the weights update from the pivot row alone:

$$w_j \leftarrow \max\left(w_j,\ \left(\frac{\alpha_{rj}}{\alpha_{rq}}\right)^2 w_q \right), \qquad w_q \leftarrow \max\left(\frac{w_q}{\alpha_{rq}^2},\ 1 \right)$$

When the weights drift too far from the truth, the framework is reset and all weights return to 1. This is the cheap approximation to steepest edge, and the reason it is affordable is that α_{rj} — the pivot row — is computed once and serves both the weight update and, as Section 7.3 shows, the reduced costs.

7.3 The two refinements that matter most here

Devex pricing. Choosing the entering variable by the most negative d_q (the textbook Dantzig rule) ignores that different columns move the solution by different distances. Devex keeps a running estimate of each column's edge norm and picks by d_j^2 / w_j — steepest descent rather than steepest slope.

Incremental pricing. Recomputing every reduced cost each iteration is a full pass over A. But after a pivot the reduced costs update from the pivot row:

$$\theta_d = \frac{d_q}{\alpha_{rq}}, \qquad d_j \leftarrow d_j - \theta_d \, \alpha_{rj}$$

and α_{rj} is exactly what the Devex update already computes. So two passes over the matrix become one. Measured: **0.80× the time** across the Netlib set, with `degen3` going from 85.2 s to 66.2 s.

7.4 The dual simplex — the one that actually runs the MILP

The primal simplex keeps x feasible and works toward dual feasibility. The **dual simplex** does the mirror: it keeps the reduced costs valid and works toward primal feasibility. Same basis machinery, the two loops swapped:

1. **Choose a leaving variable.** Pick a basic x_{B_r} that violates its bound; the size of the violation (weighted) is the pricing score.
2. **Ratio test on the row.** Compute the pivot row $\alpha_{r\cdot} = e_r^\top B^{-1} A$ and choose the entering column q minimising

   $$\frac{d_j}{\alpha_{rj}} \quad \text{over } j \text{ with the correct sign of } \alpha_{rj}$$

   The minimum is exactly the largest step that keeps every d_j on the right side of zero.
3. **Pivot.**

Why this is the node solver. After branching adds $x_j \le 3$, the parent's optimal basis is still *dual* feasible — the objective did not change, so the reduced costs did not either. Only x_j may now violate a bound. So the child starts one violated bound away from optimal, and the dual simplex repairs it in a handful of pivots.

Measured: **18,772 of 18,775** node relaxations warm start, averaging about **three pivots** each. A cold solve would be hundreds. This single fact is the reason branch-and-bound is affordable, and Section 13.1 turns on it.

And it is where the worst bug lived. Bland's anti-cycling rule — pick the lowest index among candidates — is safe in the primal, where pricing and the ratio test are separate steps, so overriding pricing changes only *which* good step you take. In the dual they are the same step: the ratio test *is* the entering choice, and it is the only thing keeping the reduced costs signed correctly. Overriding it produced `fit1p` at 33,609 against a true 9,146.38 — feasible, row violation 2.8×10^{-14}, and reported optimal. Removing the override took the suite from 13/16 to **16/16**.

7.5 Underneath everything: the LU factorisation

B^{-1} is never formed. B is factorised as $B = LU$ with a **Markowitz** pivot rule, which at each elimination step picks the entry minimising

$$(r_i - 1)(c_j - 1)$$

among entries large enough to be numerically safe ($|a_{ij}| \geq \tau \max_k |a_{kj}|$), where r_i and c_j are the remaining counts in that row and column. This is a direct trade of sparsity against stability: the product counts how many new nonzeros the elimination could create, and the threshold refuses the sparsest choice when it is too small to divide by.

Between refactorisations, each pivot is absorbed by the **product form**: the new inverse is the old one times an elementary matrix E_k, so

$$B_k^{-1} = E_k E_{k-1} \cdots E_1 U^{-1} L^{-1}$$

and FTRAN/BTRAN simply pass through the accumulated list. The list grows, so the basis is refactorised periodically — every ~100 pivots, or immediately when a stability check fails.

8. The first-order method — the piece that runs on a GPU

8.1 Why a saddle point

Write the LP as a min-max problem. For $Kx \geq q$:

$$\min_x \max_{y \geq 0} \; c^\top x + y^\top (q - Kx)$$

If x violates a constraint, y can push the inner term to $+\infty$, so the outer minimisation will not tolerate it. The saddle point of this is exactly the primal-dual optimum.

8.2 PDHG

Primal-dual hybrid gradient alternates a gradient step in each variable, each projected back onto its own constraint:

$$x^{k+1} = \Pi_{[lo,hi]} \left(x^k - \tau \left(c - K^\top y^k \right) \right)$$
$$\bar{x} = 2x^{k+1} - x^k$$
$$y^{k+1} = \Pi_{y \geq 0} \left(y^k + \sigma \left(q - K\bar{x} \right) \right)$$

Look at what is in there: **two sparse matrix-vector products** ($K^\top y$ and $K\bar{x}$), some vector arithmetic, and two clamps. No factorisation, no basis, nothing sequential.

That is the whole reason this method is on a GPU. A matrix-vector product is thousands of independent row dot-products. A clamp is elementwise. Every step is embarrassingly parallel.

The step sizes must satisfy $\tau\sigma\|K\|^2 \leq 1$; the split between them is the **primal weight** ω, with $\tau = \eta/\omega$ and $\sigma = \eta\omega$.

8.3 Restarts and Halpern

Plain PDHG converges slowly. Two accelerations:

Restarting. Periodically take the current iterate (or an average) and begin again from it. Sounds like it does nothing; it changes the convergence rate, because the method's rate depends on the distance to the optimum and restarting resets that distance.

Halpern iteration. Keep an anchor z^0 from the epoch start and pull each step toward it with a decaying weight:

$$z^{k+1} = \frac{k+1}{k+2}\, T(z^k) + \frac{1}{k+2}\, z^0$$

where T is one PDHG step. Early on the anchor pulls hard; later the pull vanishes and it becomes plain PDHG.

Reflection. Overshoot the operator's output:

$$R(z) = (1+\gamma)\, T(z) - \gamma z, \qquad \gamma \in [0, 1]$$

This costs nothing extra: $R(z) = T(z) + \gamma\,(T(z) - z)$, and $T(z) - z$ is already computed.

8.4 What this method actually is

This matters for how the project is described. Chen, Sun, Yuan, Zhang and Zhao, arXiv:2509.23903, **Proposition 3.1**: the reflected restarted Halpern PDHG with $\gamma = 1$ **is** the Halpern Peaceman–Rachford method, with the semi-proximal term taken as $T_1 = \lambda_A I - AA^*$, $\sigma = \eta/\omega$ and $\lambda_A = 1/\eta^2 \geq \|A\|^2$.

Our default reflection is 1.0. So this solver implements an **HPR method**, not a pile of enhancements to PDHG. That is worth saying precisely, because it is checkable.

The same paper measures the distance to the best implementation, HPR-LP, at accuracy 10^{-8}: on the Mittelmann LP benchmark both solve 44 of 49 with HPR-LP about 1.1× faster; on MIPLIB relaxations HPR-LP solves two more and is 1.8× faster. Their conclusion is that both are effective realisations of the same method and **the difference is implementation, not algorithm**.

8.5 Feasibility polishing

First-order methods reach moderate accuracy quickly and high accuracy slowly. For a refinery that is the wrong shape of error: a plan that violates a capacity by 0.8 units cannot be run, even if its objective looks fine.

PDLP's insight (arXiv:2501.07018 §4): a **feasibility problem** — the same constraints, no objective — is far easier for this method, because nothing is pulling against the constraints. And PDHG's iterates are non-increasing in distance to any optimal solution, so a run warm started near a good point stays near it.

So when the duality gap is already acceptable, pause and solve two subproblems warm started from where you are:

primal: $c := 0$, started from $(x, 0)$ **dual:** $q := 0$, finite bounds $:= 0$, started from $(0, y)$

The result is a point that is *almost exactly feasible* with roughly the objective you already had.

The trade is explicit and it is the right one here: it buys feasibility and pays in the duality gap. On the refinery model, at the shipped defaults:

	without polishing	with polishing
iterations	160,720	**12,800** + 1,720
capacity violation	1.46e-02	**1.28e-04**
duality gap	1.1e-09	4.5e-03

Eleven times fewer iterations for a violation 114 times smaller, at the cost of a 0.45% gap. A plan that overruns a unit is not a plan; a plan that leaves half a percent on the table is.

8.6 Scaling — the step that decides whether any of it works

A refinery model mixes barrels (10^5), fractions (10^{-2}) and rupees (10^7) in the same matrix. A first-order method takes a *single* step size for all of it, so a badly scaled matrix means the step is either far too long for one row or far too short for another. Unlike the simplex, this method has no basis to hide behind — scaling is not a nicety here, it is load-bearing.

Two diagonal matrices are found, D_r for rows and D_c for columns, and the problem is replaced by

$$\tilde{A} = D_r A D_c, \qquad \tilde{c} = D_c c, \qquad \tilde{b} = D_r b$$

with variable bounds divided by D_c. Everything is undone exactly at the end, so scaling changes the path, never the answer.

Ruiz equilibration. Repeat: divide each row by $\sqrt{\|A_{i\cdot}\|_\infty}$ and each column by $\sqrt{\|A_{\cdot j}\|_\infty}$. Each pass drives the largest absolute entry of every row and column toward 1. Ten passes are plenty.

Pock–Chambolle. A second pass tuned for exactly this algorithm, with a parameter α (we use $\alpha = 1$):

$$(D_r)_i = \frac{1}{\sqrt{\sum_j |A_{ij}|^{2-\alpha}}}, \qquad (D_c)_j = \frac{1}{\sqrt{\sum_i |A_{ij}|^{\alpha}}}$$

This is not arbitrary: it is chosen so that the step-size condition $\tau\sigma\|\tilde{A}\|^2 \le 1$ is satisfiable with a *uniform* step, which is what the algorithm actually needs.

Both are applied, Ruiz first. On the refinery model the matrix norm drops by more than an order of magnitude, and with it the number of iterations.

8.7 When to stop — the termination test

The solver may not stop when "the numbers stop moving". It stops when it can show the three conditions of optimality are met to a stated tolerance. Unscale first, then measure on the original problem — a small residual in scaled space can be a large one in barrels.

Primal residual — how far the constraints are from being satisfied:

$$r_p = \left\| \Pi_{[l,u]}(Ax) - Ax \right\|_2$$

Dual residual — how far the reduced costs are from being consistent with the bounds the variables actually sit at:

$$r_d = \left\| c - A^\top y - \lambda \right\|_2$$

where λ is the part of the reduced cost that a variable at a bound is allowed to absorb.

Duality gap — the two objectives must meet (Section 4.2):

$$g = \left| \, c^\top x - (q^\top y + \text{bound terms}) \, \right|$$

Each is tested *relative*, not absolute, because "violation of 0.001" means nothing until you know whether the row's right-hand side is 1 or 10^6:

$$r_p \le \varepsilon \, (1 + \|b\|), \qquad r_d \le \varepsilon \, (1 + \|c\|), \qquad g \le \varepsilon \, (1 + |c^\top x| + |b^\top y|)$$

The gap tolerance is separately settable (`--gap-tol`), and that flag exists for an honest reason: the gap is the slowest of the three to close, and a planner usually wants a strictly runnable plan sooner rather than a provably-last-rupee plan later.

8.8 Proving there is no answer

A solver that only ever says "here is the optimum" is half a tool. If a planner over-constrains a model, the useful reply is *"these constraints cannot all hold"* — proved, not guessed from a stall.

Farkas' lemma gives the proof object. Exactly one of the following holds:

$$\exists \, x \ge 0 : Ax = b \qquad \text{or} \qquad \exists \, y : A^\top y \le 0 \;\text{ and }\; b^\top y > 0$$

The second y is a **certificate**: any non-negative combination of the rows that produces a contradiction. It can be checked in one matrix-vector product by someone who does not trust the solver at all.

The first-order method produces these naturally. When a problem is infeasible its iterates diverge, but the *direction* of divergence converges — the normalised difference $ (z^{k+1} - z^{k})/|z^{k+1}-z^k| $ settles onto the certificate. So the solver watches the difference direction alongside the iterate and, when that direction satisfies the Farkas conditions to tolerance, reports infeasibility with the

certificate attached. Unboundedness is the mirror image, with the primal difference giving a ray of improvement.

8.9 Step size, primal weight, restarts — the three tuned quantities

These three are where a first-order LP solver is won or lost, and each is one formula.

Step size. The theory requires $\tau\sigma\|A\|^2 \le 1$. Two ways to get there:

Adaptive — try a step, measure whether the movement was consistent with the local curvature, and accept or shrink:

$$\bar{\eta} = \frac{\|\Delta z\|_\omega^2}{2\,|\Delta y^\top A\,\Delta x|}$$

then accept if $\eta \le \bar{\eta}$ and propose the next η from $\bar{\eta}$ with a decaying exploration term. Costs one extra product per rejected step.

Constant — estimate $\|A\|_2$ once by power iteration and set

$$\eta = \frac{\theta}{\|A\|_2}$$

cuPDLPx uses $\theta = 0.998$. Both were implemented and swept over Netlib:

θ	instances solved (of 88)
adaptive	76
0.90	**75**
0.95	74
0.998	74

The literature's value is not the best value *here*, and adaptive is best of all on this set. The shipped default is adaptive, with the constant path available and $\theta = 0.90$ if it is used. Sweeping was cheaper than believing.

Primal weight. ω splits the single step between primal and dual: $\tau = \eta/\omega$, $\sigma = \eta\omega$. It should track the relative size of movement on the two sides, so the classic update is a smoothed ratio in log space:

$$\omega^{k+1} = \exp\Big(\theta_\omega \log\frac{\|\Delta y\|}{\|\Delta x\|} + (1-\theta_\omega)\log\omega^k\Big)$$

cuPDLPx replaces this with a **PID controller** on the log-ratio error $e = \log\|\Delta y\| - \log\|\Delta x\|$:

$$\log\omega^{k+1} = \log\omega^k + K_p\,e + K_i\sum e + K_d\,(e - e_{\text{prev}})$$

Swept here: $K_p = 0.5$, $K_i = 0$, $K_d = 0.3$. The integral term was measured and set to zero — it accumulates and overshoots on this set.

Restarts. cuPDLP's rule restarts the epoch when any of three conditions fires, on a normalised duality gap μ measured over the epoch:

$$\begin{aligned} \text{sufficient decay:} \quad & \mu_c \le 0.2\,\mu_0 \\ \text{necessary decay, and no further progress:} \quad & \mu_c \le 0.8\,\mu_0 \ \text{ and } \ \mu_c > \mu_c^{\text{prev}} \\ \text{epoch already long:} \quad & k \ge 0.36\,K \end{aligned}$$

cuPDLPx simplifies this to a **fixed-point residual** test — restart when $\|z - T(z)\|$ has fallen by a fixed factor since the epoch began. Cheaper, since $z - T(z)$ is already computed for the reflection, and it does not need the gap. Both are implemented; the fixed-point rule is the default.

9. Branch and cut, concretely

9.1 Cuts

A **cutting plane** is an inequality valid for every integer feasible point but violated by the current fractional relaxation solution. Adding it tightens the bound without removing any answer.

Cover cuts. For a knapsack row $\sum_j a_j x_j \le b$ with binary x_j and $a_j > 0$, a set C is a *cover* if $\sum_{j\in C} a_j > b$ — the items cannot all be taken. Therefore

$$\sum_{j\in C} x_j \le |C| - 1$$

MIR (mixed integer rounding). For $\sum_j a_j y_j \le b$ with $y \ge 0$ and y_j integer for $j \in I$, let $f = b - \lfloor b \rfloor$ and $f_j = a_j - \lfloor a_j \rfloor$. Then

$$\sum_{j\in I} \left(\lfloor a_j \rfloor + \frac{\max(0, f_j - f)}{1 - f} \right) y_j \ + \ \sum_{j\notin I} \frac{\min(0, a_j)}{1 - f}\, y_j \ \le \ \lfloor b \rfloor$$

Gomory mixed-integer cuts apply the same rounding to a row of the simplex tableau rather than a row of the model, which lets them see combinations the model's own rows cannot.

9.2 Reduced-cost fixing

Once there is an incumbent with objective z_{inc} and a node with bound z_{node}, a nonbasic variable at its lower bound with reduced cost $d_j > 0$ costs at least d_j per unit moved. So moving it further than

$$x_j \le l_j + \frac{z_{\text{inc}} - z_{\text{node}}}{d_j}$$

cannot beat the incumbent. Tighten the bound for the children.

Measured: `gt2` from 7,901 nodes to **1,167**.

9.3 Node propagation

After branching pins $x_j \le 3$, interval arithmetic on each row can often tighten other variables — and sometimes prove the child infeasible before its LP is ever solved. For a row $\sum_j a_j x_j \ge q$, the maximum activity is

$$\text{maxact} = \sum_{a_j>0} a_j\, hi_j + \sum_{a_j<0} a_j\, lo_j$$

and if maxact $< q$ the row cannot be satisfied.

Measured: `flugpl` from 28,917 nodes to **477**.

9.4 Reliability branching

Pseudocost branching estimates, from history, how much the bound rises when a variable is branched. But a variable never branched has no history and gets the same optimistic guess as every other — so decisions near the root, which shape the whole tree, are made blind.

Reliability branching strong-branches (actually solves both children, with a small iteration budget) until a variable's pseudocost is trustworthy, then trusts it.

The measured subtlety: it must be **capped by depth**. Unlimited, it made `gt2` nearly three times *worse* (783 → 2,225 nodes). Capped at depth 10 it is 194. Near the root a decision shapes the tree; deep down it settles a subtree about to be pruned anyway, and the greedy one-level-ahead choice is not the one that makes the smallest tree.

10. QP by ADMM, written out

10.1 The splitting

The problem is

$$\min_x \tfrac{1}{2} x^\top Q x + c^\top x \quad \text{s.t.} \quad l \le Ax \le u$$

The difficulty is that x appears in both a smooth objective and a hard constraint. **Splitting** separates them: introduce a copy $z = Ax$ and put the constraint on the copy.

$$\min_{x,z} \ \tfrac{1}{2} x^\top Q x + c^\top x + \mathcal{J}_{[l,u]}(z) \quad \text{s.t.} \quad Ax = z$$

where $\mathcal{J}$ is zero inside the box and $+\infty$ outside. Now each half is easy on its own: the x half is a linear solve, the z half is a clamp. Alternating direction method of multipliers alternates them, with a dual variable y enforcing the link:

$$
\begin{aligned}
(\tilde{x}^{k+1}, \tilde{z}^{k+1}) &\leftarrow \text{solve the linear system below} \\
x^{k+1} &= \alpha\tilde{x}^{k+1} + (1-\alpha)x^k \\
z^{k+1} &= \Pi_{[l,u]}(\alpha\tilde{z}^{k+1} + (1-\alpha)z^k + \rho^{-1}y^k) \\
y^{k+1} &= y^k + \rho(\alpha\tilde{z}^{k+1} + (1-\alpha)z^k - z^{k+1})
\end{aligned}
$$

α is relaxation (1.6 works well), ρ the penalty on the link.

10.2 The linear system, and why it always factorises

Every iteration solves the same KKT system:

$$
\begin{bmatrix} Q + \sigma I & A^\top \\ A & -\rho^{-1}I \end{bmatrix} \begin{bmatrix} \tilde{x} \\ \nu \end{bmatrix} = \begin{bmatrix} \sigma x^k - c \\ z^k - \rho^{-1}y^k \end{bmatrix}
$$

Note what changes between iterations: **only the right-hand side.** The matrix is fixed as long as ρ is. So it is factorised once and every subsequent iteration is two triangular solves.

The matrix is **quasi-definite**: the $(1,1)$ block $Q + \sigma I$ is positive definite (because $\sigma > 0$, even if Q is only positive *semi*-definite) and the $(2,2)$ block $-\rho^{-1}I$ is negative definite.

Vanderbei's theorem: every symmetric permutation of a quasi-definite matrix has an $LDL^\top$ factorisation. That is a strong statement — it means no pivot will ever be zero, no matter what ordering is chosen. So the ordering can be picked *purely* to minimise fill (AMD on the sparsity pattern), computed once, and reused. No numerical pivoting, no symbolic re-analysis per iteration.

That is the entire reason this method is fast, and it is why σ exists at all — it is not regularisation for its own sake, it is what buys the theorem.

10.3 The factorisation itself

Up-looking sparse $LDL^\top$ (Davis, Algorithm 849). Two phases:

Symbolic. Build the **elimination tree**: parent(k) is the row of the first off-diagonal nonzero in column k of L. Walking from a nonzero of K up this tree to the already-computed part gives exactly the nonzero pattern of that row of L — without any floating point. This yields the column counts, so the numeric phase allocates once.

Numeric. For each row k, solve a sparse triangular system against the rows already computed, then

$$
D_k = K_{kk} - \sum_{j \in \text{pattern}} L_{kj}^2 D_j
$$

Measured against solving the same system iteratively with conjugate gradients: **1.52× faster**, and without CG's dependence on conditioning.

10.4 Adaptive ρ, and where it breaks

ρ balances how hard the link is enforced. Too small and the copy drifts from Ax; too large and the objective is ignored. The standard rule rescales it from the ratio of the two residuals:

$$\rho \leftarrow \rho\sqrt{\frac{\hat{r}_{\text{prim}}}{\hat{r}_{\text{dual}}}}, \qquad \hat{r}_{\text{prim}} = \frac{\|r_{\text{prim}}\|}{\max(\|Ax\|, \|z\|)}, \quad \hat{r}_{\text{dual}} = \frac{\|r_{\text{dual}}\|}{\max(\|Qx\|, \|A^\top y\|, \|c\|)}$$

and refactorises when it changes (hence the gate: only rescale when the ratio exceeds 5, so the factorisation is not thrown away every iteration).

This is the part that fails on five instances, four of them the PRIMALC family. Traced: ρ falls from 10^{-1} to 1.9×10^{-5} within 500 iterations, and the same gate that stops it oscillating also stops it climbing back. Section 13.8 records the two fixes that followed from that diagnosis and the measurements that killed both.

10.5 Polishing

ADMM converges to moderate accuracy quickly, like every first-order method. But once it has converged, the *active set* — which constraints are tight — is usually exactly right even when the numbers are not. So: guess the active set from the converged y, form the reduced KKT system with only those rows as equalities, and solve it directly. If the result satisfies the original constraints, keep it. It is the same trade as feasibility polishing in Section 8.5 — use the cheap method to find the *structure*, then solve the small exact problem that structure implies.

11. Presolve, with the algebra

Presolve shrinks the model before it is ever solved. On Netlib it removes about **19% of rows and 12% of columns** without changing a single answer. It is the cheapest speedup in the whole solver, and the most dangerous — every reduction is a claim that some solutions can be discarded, and a wrong claim discards the answer.

Eleven reductions are implemented. Here is what each actually computes.

Empty row. No entries. Either the bounds contain zero (drop it) or they do not (the model is infeasible, and this is a proof).

Singleton row. One entry: $a\,x_j \in [l, u]$ is just a bound on x_j. Intersect it with the existing bound:

$$lo_j \leftarrow \max\Big(lo_j,\ \tfrac{l}{a}\Big), \qquad hi_j \leftarrow \min\Big(hi_j,\ \tfrac{u}{a}\Big)$$

with the two swapped when $a < 0$. Then delete the row. Postsolve must restore the row's dual, which is $y = d_j/a$ where d_j is the reduced cost the reduced problem reports for that column.

Empty column. x_j appears in no row, so only the objective decides it: fix it at whichever bound c_j prefers. Unbounded in that direction means the LP is unbounded.

Fixed column. $lo_j = hi_j$. Substitute the value into every row's bounds and delete the column:

$$l_i \leftarrow l_i - A_{ij}\, lo_j, \qquad u_i \leftarrow u_i - A_{ij}\, lo_j$$

Forcing row. Compute the row's activity range from the variable bounds:

$$\text{minact} = \sum_{a_{ij}>0} a_{ij}\, lo_j + \sum_{a_{ij}<0} a_{ij}\, hi_j, \qquad \text{maxact} = \sum_{a_{ij}>0} a_{ij}\, hi_j + \sum_{a_{ij}<0} a_{ij}\, lo_j$$

If minact $= u_i$, then *every* variable in the row must be at the bound that achieved the minimum — there is no slack anywhere. Fix them all and drop the row. If minact $> u_i$ or maxact $< l_i$, the model is infeasible. And if the range lies strictly inside $[l_i, u_i]$, the row can never bind and is **redundant** — delete it.

This is the reduction that caused the first wrong answer. My own 10^{-9} padding on the bounds manufactured a forcing row where none existed. Fixed by separating two tolerances: a tight one to *fire* a reduction, a looser one to *declare infeasibility*, and declining to act in the band between them.

Doubleton equation. A row $ax_i + bx_j = c$ gives $x_i = (c - bx_j)/a$. Substituting into every other row k containing x_i:

$$A_{ki}x_i = \frac{A_{ki}c}{a} - \frac{A_{ki}b}{a}x_j$$

so row k's bounds shift by $-A_{ki}c/a$ and x_j's coefficient there changes by $-A_{ki}b/a$. The objective picks up c_i the same way.

Only applied when x_i is **implied free** — the row plus x_j's bounds already confine x_i strictly inside its own bounds — because then x_i can never sit *at* a bound, so its reduced cost is zero, and the eliminated row's dual comes back exactly:

$$y_r = \frac{c_i - \sum_{k\neq r} A_{ki}\, y_k}{a}$$

Without implied-freeness the dual is not recoverable and the shadow prices of Section 4.4 would be quietly wrong.

Two bugs lived here. The staleness guard ran *after* the liveness check — but a column that has been substituted away is also a dead column, so the guard never fired. And a column that survived the substitution could be eliminated by a later reduction, leaving `stocfor2` with 189 units of activity sitting outside the model.

Dual fixing. Count each column's **locks**: how many rows could be violated by moving x_j up, and how many by moving it down. If nothing can be violated by moving it *down* and $c_j \geq 0$, then some optimal solution has x_j at its lower bound — pushing it down is free and never worse. Fix it there. The mirror case fixes at the upper bound.

This generalises the empty-column rule: that one fires when *nothing at all* can stop the column; this one when nothing can stop it *in the direction the objective already prefers.*

Free/implied-free column substitution, **duplicate rows**, and **singleton-column-in-equality** round out the eleven.

11.1 Postsolve is a replay, not an inverse

Reductions are pushed onto a stack as they fire. Postsolve pops them in reverse, each one reconstructing what it removed. Primal recovery is mechanical.

Dual recovery is not. Some reductions invert exactly (singleton row, doubleton with implied-freeness); some do not. The implementation carries a `dual_is_exact()` flag per reduction, and when a run contains an inexact one the solver *says so* rather than reporting a shadow price it cannot stand behind. Given Section 4.4 — a planner may be sizing capital on these numbers — that is not pedantry.

12. How the pieces fit together

For an **LP**: read → presolve → scale → solve (simplex, or first-order on the GPU) → unscale → postsolve → verify optimality conditions → report x, y, and the reduced costs.

For a **MILP**: presolve once at the root, solve the root relaxation, then repeatedly — select a node, propagate bounds into it, solve its relaxation with the **dual simplex warm started from the parent** (Section 7.4), separate cuts if it is worth it, apply reduced-cost fixing against the incumbent, and either prune it or branch. The LP solver is called tens of thousands of times; everything in Section 9 exists to reduce that count.

For a **QP**: presolve → scale → ADMM with a single cached $LDL^\top$ → polish.

The GPU sits underneath as a backend: the same first-order algorithm, with the vectors resident on the device and the matrix-vector products, reductions and clamps as kernels. Measured **2.70×–7.09×** over the same algorithm on CPU on a Tesla T4. Two things learned the hard way — `cudaMalloc` does *not* zero its memory where the CPU allocator value-initialises (Section 14), and on one instance **79% of solve time was outside the kernels**, in the convergence check that still runs on the host. That is the top remaining performance item.

Part IV — Every option that was rejected, and why

This is the part that is usually missing from a project write-up. Each of these is standard, respectable and in the textbooks. Each was measured against *this* instance set and each lost.

13. Rejected, with the numbers

13.1 Interior point methods — the full argument

The plan disposed of this in eight words. Here it is from every side.

Against, the obvious way. An IPM needs a sparse Cholesky of $ADA^\top$ (or an $LDL^\top$ of the augmented system) at *every* iteration, with a fill-reducing ordering. Large machinery that nothing else reuses.

Against, the stronger way — it serves neither goal:

- **MILP needs warm starts.** The dual simplex restarts from the parent's basis and finishes a child in about **three pivots** — 18,772 of 18,775 relaxations warm start here. An IPM does not warm start usefully, so every node solves from scratch. Branch-and-bound as built would not survive that.
- **The GPU needs matrix-vector products.** An IPM's time is in the sparse factorisation, which is the part that parallelises worst.

For — and this is real. First-order methods converge *linearly*, which is exactly where this solver is weakest. It is why `--gap-tol` exists and why feasibility polishing had to be built. A second-order method gives high accuracy natively. The recent literature makes this argument explicitly.

And the ground has moved:

- NVIDIA shipped **cuDSS**, a GPU direct sparse solver with Cholesky, $LDL^\top$ and LU — the factorisation an IPM needs is now on the device.
- **Condensed-space IPM** reshapes the KKT system into symmetric positive definite form, which factorises far better on a GPU (arXiv:2405.14236).

Does the decision still hold? Yes — for a different reason than originally given. Not because "IPM cannot work on a GPU" (less true every year), but because:

1. The accuracy an IPM would add is **already covered by the simplex**, which gets 16 of 16 Netlib instances with published optima.
2. Even with cuDSS, reported gains for fully GPU-based interior-point LP solvers **remain modest** — the sparse factorisation is still the bottleneck.
3. It still would not warm start for branch-and-bound.

Where the honest concession is: QP. For LP, an IPM overlaps methods we already have. For MILP, it is actively worse. But commercial solvers use barrier for QP, and our five QP failures are step-size tuning failures that an IPM would simply not have — it takes Newton steps and has no step-size parameter to mistune. *"IPM would have been better for QP"* is a true statement. It was not built because QP is the smallest of the three components and the machinery is three to four weeks.

What would change it: the refinery model growing to where simplex becomes impractical *and* tight accuracy is required. Written down so the decision gets revisited on evidence.

13.2 Hypersparsity

Hall and McKinnon report a **5.2× mean speedup** from exploiting it. Their criterion: an instance is hyper-sparse when more than 60% of FTRAN/BTRAN results have density under 10%.

Measured here over 21 Netlib instances:

	ftran <10%	btran <10%	combined
czprob	99.6%	43.2%	**75.6%**
80bau3b	80.2%	65.0%	**72.9%**
pilot87	3.4%	4.7%	4.1%
degen3	6.1%	10.5%	8.5%
all 21	**13.1%**	**17.5%**	**15.4%**

Four of twenty-one clear the threshold. Their 5.2× came from a test set *selected* for the property (KEN-18, PDS-20, STOCFOR3 — network-structured). Netlib is not that. Six to eight weeks of work to help four instances.

13.3 Forrest–Tomlin updates

A sampling profile of `degen3`, the slowest instance at 66 s, puts 67% of samples in one place and `LuFactor::factorize` at **11 samples out of ~4,500**. Basis updates are not where the time goes. Aimed at something that is not hot.

13.4 Bound-flipping ratio test

Needs boxed columns. Measured: `pilot87` 36.8%, `80bau3b` 35.6%, `fit1p` 23.8% — and **0%** for afiro, sctap1, degen3, 25fv47, woodw, stocfor2 and maros-r7.

13.5 Coefficient tightening

Implemented in full. Fires **three times** across seven MIPLIB instances, all on one, and moves that instance's root bound from 6875.00000004 to 6875.00000002 — which is noise. Kept in the source, switched off, with the measurement beside it.

13.6 Parallel column merging

20% of Netlib columns are parallel to another — 31,954 of 159,369, all continuous, which is the safe case. Compelling until the merge condition is applied: it also needs the objective in the same ratio, which cuts it to **1,471**. `standata` goes from 606 parallel to 12 mergeable.

0.9% of columns, against needing a new postsolve entry kind that writes two columns from one merged value — the most dangerous part of this codebase to extend. Measured, not built.

13.7 Harris ratio test without EXPAND

Three instances better, six worse, and `blend` stopped solving entirely.

13.8 Two fixes for a QP convergence failure

Five QP instances fail, four of them the PRIMALC family whose DUALC counterparts all solve. Diagnosis was clear: the adaptive step-size rule drives ρ from 10^{-1} to 1.9×10^{-5} within 500 iterations and the gate that stops it oscillating also stops it recovering.

Two fixes follow from that diagnosis. **Neither worked.** Limiting the drift made things monotonically worse (35/40 unlimited, 33 at a factor of 100, 30 at 10). Relaxing the gate after a long stall changed nothing at all. Both reverted, with the measurements kept where the option would have been.

Part V — What went wrong, and what it taught

14. Six times the solver was confidently wrong

The plan's risk register had eleven risks and ten were about *finishing on time*. That risk never materialised. This one did, repeatedly.

Presolve declared a feasible model infeasible. My own 1e-9 bound padding manufactured forcing rows. Fixed by using two tolerances — one to fire a reduction, a looser one to declare infeasibility — and declining to reduce in between.

The dual simplex reported a wrong answer as optimal. `fit1p` at 33,609 against a true 9,146.38, feasible, row violation 2.8e-14. Cause: Bland's anti-cycling rule was overriding the dual's ratio test. In the primal, pricing and the ratio test are separate steps so overriding pricing is safe; in the dual the ratio test *is* the entering choice and the only thing keeping reduced costs on the right side of zero. Removing the override: 13/16 → **16/16**.

A cover cut removed feasible points. The separator inferred whether an item was complemented by testing `slack == x_j` — true for complemented items, false otherwise, *except* at $x_j = 0.5$ where $1 - x_j$ is also 0.5 and everything reads as complemented. A simplex vertex can sit a binary at exactly 0.5. 426 separations at random interior points never landed there.

An $LDL^\top$ silently became a diagonal factorisation. Handed the upper triangle where the algorithm needs the lower. Every entry skipped, elimination tree empty. The diagonal test passed at 1e-14 and said nothing.

The GPU never uploaded the dual iterate. Harmless while everything started at zero — and silently fatal for any warm start, because on CUDA the loop began from whatever `cudaMalloc` returned. The CPU allocator zeroes; CUDA does not.

`fiber` returned a proved optimum 60.8% wrong. 652,748.78 against a true 405,935.18, with a matching dual bound and no complaint. Two separate bugs:

1. A bound crossing of 1.78×10^{-15} — the last bit of a double — treated as proof that a box was empty. Bound propagation tolerates crossings up to 10^{-7}; the infeasibility check tolerated nothing. **The two disagreed about what an empty box is** and the answer fell through the gap.
2. A cover cut derived from an *earlier cut* rather than a model row. Each cut family was valid alone — only the combination broke, because only the combined run reached the point that produced the bad cut.

15. The tools that exist because something got through

- A **dual-feasibility check** the simplex runs on itself before claiming optimality. It found the Bland bug on its first run.
- **Cut validity by enumeration** over small programs, separating at simplex vertices as well as random points.
- **An $LDL^\top$ test** that builds K, picks x, forms $b = Kx$ and compares — never a residual the factorisation computed about itself.
- **A survey against 103 published optima** instead of seven. It found the `fiber` wrong answer within minutes of existing.

- **A debug-solution tracker.** Hand the solver an answer known to be correct; every point that can discard a node first checks whether that answer is inside it, and names the first prune that throws it away. SCIP carries the same facility. It found both `fiber` bugs in minutes after four layers of manual elimination had found neither.

16. Traps in measuring, not in the code

- **`zsh` does not word-split unquoted variables.** A variable holding two flags arrives as one argument, the program rejects it, and the checker silently reads a stale file. Cost three separate debugging sessions.
- **A measurement under load is not a measurement.** Twice a benchmark reported a regression that did not exist. Both times the change was a pure improvement.
- **A binary linked against a static library does not relink when the library is rebuilt.** Twenty minutes reading correct code that was not being executed.

Part VI — Where it stands and what is next

17. Verified against published answers

- Reader matches HiGHS on all 88 Netlib instances, 1.4–1.5× faster.
- Primal and dual simplex both correct on all 16 Netlib instances with published optima.
- Presolve removes ~19% of rows and ~12% of columns without changing any answer, and recovers duals as well as primals.
- GPU measured 2.70×–7.09× over the same algorithm on CPU (Tesla T4).
- Branch-and-cut proves optimality on the smaller MIPLIB instances; on several it does not prove, the *solution* is already the published optimum.
- QP solves 35 of the 40 smallest Maros–Meszaros instances.

18. The benchmark to be judged against

Mittelmann's LPfeas benchmark is where cuPDLPx and HPR-LP — the two papers this method comes from — are actually scored. Over 65 problems, cuPDLPx solves 57, HiGHS 55, and OR-Tools' PDLP 50.

That last number is worth sitting with. **OR-Tools' PDLP is the same algorithm family implemented here, and it solves the fewest of any code except KNITRO.** The method is not automatically good. The implementation is most of it.

Eight of its forty public instances are already here. Reporting solved-or-not at 10^{-6} against that published table is a comparison a reader can check.

19. What is next, in order

1. **Verify on a GPU.** Nothing built since the last run has been tested on hardware, and the CPU hides a class of bug that only appears on the device. This closes a risk; it is not a feature.
2. **Re-fit the branch-and-bound constants against 103 instances, not seven.** Every one of them was chosen against seven, and the wider set found a wrong answer within minutes.
3. **Move the convergence check onto the device.** On one instance 79% of solve time is outside the kernels. Half done.
4. **Report against the LPfeas table.**
5. **Continuous integration.** The test suite is good; nothing runs it automatically.

Deliberately **not** on that list, each for a measured reason in Part IV: interior point, hypersparsity, Forrest–Tomlin, bound-flipping ratio test, parallel columns, coefficient tightening.

20. The one thing worth taking away

The algorithms were not the hard part. They are in papers and they work.

What took the time was finding out **when the solver was quietly wrong** — and building the things that make that visible.

A slow solver tells you it is slow. A wrong one tells you nothing: it hands back a confident number with a matching bound and no complaint. On `fiber` it was wrong by 60% and looked completely healthy.

Every checking tool in Section 15 exists because something got through.